1. Vulnerability Assessment Report Structure

- **Executive Summary:** A high-level overview of the assessment, overall risk, important findings, and business impact.
- **Findings:** Details of each identified vulnerability, including its description, affected area, evidence, and risk rating.
- **Recommendations:** Clear actions that the development or operations team should take to reduce or remove each risk.
- **Appendix:** Supporting information such as assessment scope, testing approach, scan evidence, definitions, and additional technical details.

2. Executive Summary – Food Delivery Application

A vulnerability assessment was conducted on a fictional food delivery application named **QuickBite**. The assessment identified security weaknesses that could affect customer information and order privacy. The overall risk is considered **High** because some weaknesses may allow unauthorized access to sensitive customer or business data. The key findings include SQL Injection in an application input and Insecure Direct Object Reference (IDOR) in order details. If left unresolved, these issues could lead to data exposure, loss of customer trust, and business or financial impact. The development team should prioritize the recommended fixes and perform a follow-up security assessment.

3. Vulnerability Findings – E-commerce Website

Finding 1: SQL Injection

- **Description:** An application input may handle user-supplied data unsafely, allowing specially crafted input to alter a database query.
- **Risk rating:** High.
- **Potential impact:** An attacker may access, modify, or delete database information, potentially exposing customer or order records.

Sample Evidence • SQL Injection
Test area: Product search input
Illustrative test input: ' OR '1'='1
Observed sample behaviour: The application returned unexpected results instead of safely rejecting or handling the input.
Interpretation: This behaviour may indicate unsafe query construction and requires developer verification.

4. Vulnerability Findings – Continued

Finding 2: Insecure Direct Object Reference (IDOR)

- **Description:** The application may use a predictable order identifier without properly checking whether the logged-in user owns that order.
- **Risk rating:** Medium.
- **Potential impact:** A user may be able to view another customer's order details, delivery address, or other private information.

Sample Evidence • IDOR
Test area: Order details page
Illustrative request: GET /orders/1002
Sample observation: Changing the order identifier returned order information without an effective ownership check.
Interpretation: The server should verify that the authenticated user is authorized to access the requested order.

5. Recommendations

Finding	Recommended action
SQL Injection	Use parameterized queries or prepared statements for all database operations. Validate input on the server side and avoid building SQL queries by joining raw user input.
IDOR	Enforce server-side authorization checks for every order request. Confirm that the requested order belongs to the authenticated user, and use deny-by-default access controls.

6. Conclusion

The assessment identified two important application security issues: SQL Injection and IDOR. The development team should address the High-risk SQL Injection finding first, followed by the IDOR authorization weakness. After applying the fixes, the application should be retested to confirm that the vulnerabilities have been resolved.